POLITECNICO DI TORINO

Large Language Models for Software Engineering · Academic Year 2025 / 2026

Pure-Gemma Evolution Matrix

Agentic Workflows for Python Code Generation • Politecnico di Torino • April 2026

Politecnico di Torino — Large Language Models for Software Engineering (2025 / 2026)
Johnprice Osagie · Deniz Eren Gencay
Matteo Gastaldello · Erestina Vreto

Subtitle: Agentic Workflows for Python Code Generation
Focus: scaling laws within a single model family (Gemma)
Evaluation scope: 5 models × 11 agents × 4 prompts on EvalPlus (HumanEval+)

Torino · April 2026

Tech Stack & Dataset

- Orchestration: LangGraph — stateful multi-agent workflows
- LLM backend: Google Vertex AI / AI Studio (Gemma family)
- Data engineering: pandas — consolidated 220-row results matrix
- Evaluation framework: EvalPlus (augmented HumanEval)
- Custom instrumentation: sys.settrace + AST parsing
- Multi-process isolation with hardware timeouts (kill-switch)
- Why EvalPlus over HumanEval: ~80× more test cases per task
- EvalPlus catches memorized solutions that fail edge cases
- Adds stress tests: boundary ints, empty lists, unicode, large inputs
- Standard coverage tools unreliable on Windows → custom AST-Trace
- Tracks real bytecode execution, not just line numbers
- Identifies branch points via AST (if / elif / for / while)
- 25-task subset: pure-logic problems — no I/O, no networking
- Selection rationale: isolate reasoning from API knowledge
- Reproducibility: fixed seeds + deterministic temperature
- Visualisations: Matplotlib / Seaborn
- Code in src/, results in results/<model>/, aggregates in reports/
- Requirements pinned in requirements.txt

Prompting Techniques

- Four prompting styles per (model × agent) combination
- Zero-shot — plain task description, no example
- Few-shot — 2 solved HumanEval examples as priming
- Chain-of-Thought (CoT) — "Let's think step by step"
- SCoT (Structured CoT) — reasoning → plan → code
- Routed through apply_reasoning_style() in src/utils/prompts.py
- Goal: isolate prompting contribution vs. agent architecture
- Finding: few-shot dominates 4B; CoT helps 27B / 4-31B
- Finding: 12B is hurt by CoT — verbose but wrong
- 4B outputs cleanest code blocks — lowest parser-error rate
- 27B+ adds prose preamble → needs regex stripping
- Temperature: 0.2 deterministic • 0.7 for swarm diversity
- Max tokens: 1024 (sufficient for HumanEval scale)
- Standardized system prompt for fairness across agents
- Ablation: same agent + different prompt → up to 0.30 FC delta
- Few-shot risk: example leakage / pattern over-fit
- CoT risk: hallucinated reasoning justifies wrong answer
- Prompting is no substitute for agent feedback loops

Agents Overview

- 11 distinct agentic workflows implemented under src/agents/
- Baseline — single LLM call, no loop (control condition)
- Generation agents: Swarm, Atomic Swarm, CoA, SOA
- Validation agents: Consensus, Self-Healing, Actor-Critic
- Adversarial agents: Adversarial, Competitive
- Hybrid agents: combine generation + validation
- Common state managed by LangGraph StateGraph
- Conditional edges transition on test pass / fail
- Max iteration cap (5–10 rounds) prevents infinite loops
- Matrix: 5 models × 11 agents × 4 prompts = 220 configs

Pseudocode — main feature

```
# baseline.py — control condition
def run(task, model):
    return model.generate(task.prompt)
```

Swarm, Atomic Swarm, Hybrid

- Swarm — N parallel generators, aggregate by vote
- Atomic Swarm — workers also emit their own unit tests
- Cross-validation: run worker_i code against worker_j tests
- Hybrid — Swarm population, then Self-Healing on failures
- Aggregation: majority vote / test-based / LCS
- Token cost is linear in N — dominant expense
- Optimal N: 10 for 4B, 5 for 27B (diminishing returns)
- Risk: correlated hallucinations across workers

Pseudocode — main feature

```
def swarm(task, model, N=10):
    candidates = [model.generate(task.prompt) for _ in range(N)]
    return aggregate(candidates, strategy="vote")

def hybrid(task, model):
    code = swarm(task, model)
    while not passes(code) and rounds < 5:
        code = self_heal(code, errors)
    return code
```

Consensus, Self-Healing, Adversarial

- Consensus — 3 proposers + 1 judge model
- Judge ranks candidates on correctness + clarity
- Self-Healing — generate → test → feedback → regenerate
- Max 5 healing rounds; abort if no diff from previous
- Adversarial — Tester defends, Hacker breaks
- Hacker proposes inputs that fail the candidate
- Useful for surfacing edge-case blind spots
- Self-Healing cheapest in tokens; Consensus highest ceiling

Pseudocode — main feature

```
def consensus(task, proposers, judge):
    cands = [p.generate(task.prompt) for p in proposers]
    return judge.select(cands, criteria="correctness")

def self_heal(task, model, max_rounds=5):
    code = model.generate(task.prompt)
    for _ in range(max_rounds):
        if passes(code): return code
        code = model.generate(task.prompt, errors=run(code))
    return code
```

CoA, SOA, Actor-Critic, Competitive

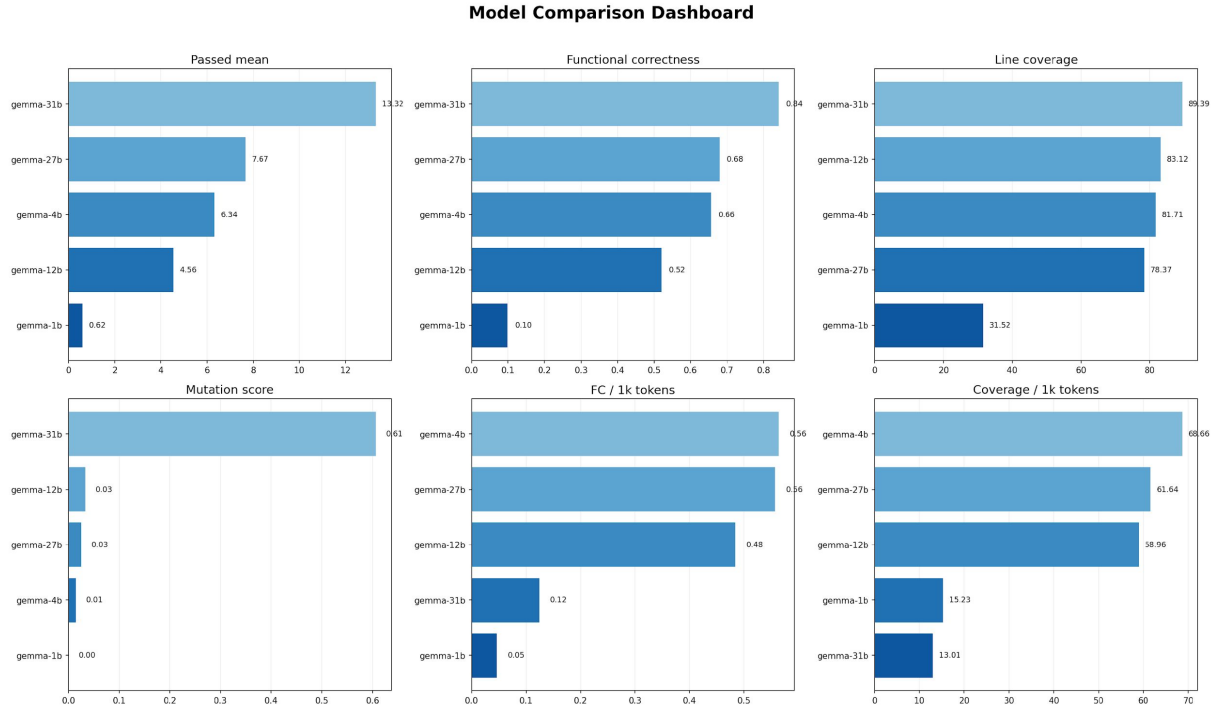
- CoA (Chain-of-Agents) — Planner → Coder → Tester → Fixer
- Each stage passes structured output to the next
- SOA (Society of Agents) — Architect, Coder, Reviewer dialogue
- Actor-Critic — generator + persistent critic loop
- Loop until Critic score > threshold or max iters reached
- Competitive — two generators judged head-to-head
- Strength: explicit quality signal at every step
- Weakness: errors compound; Critic must outrank Actor

Pseudocode — main feature

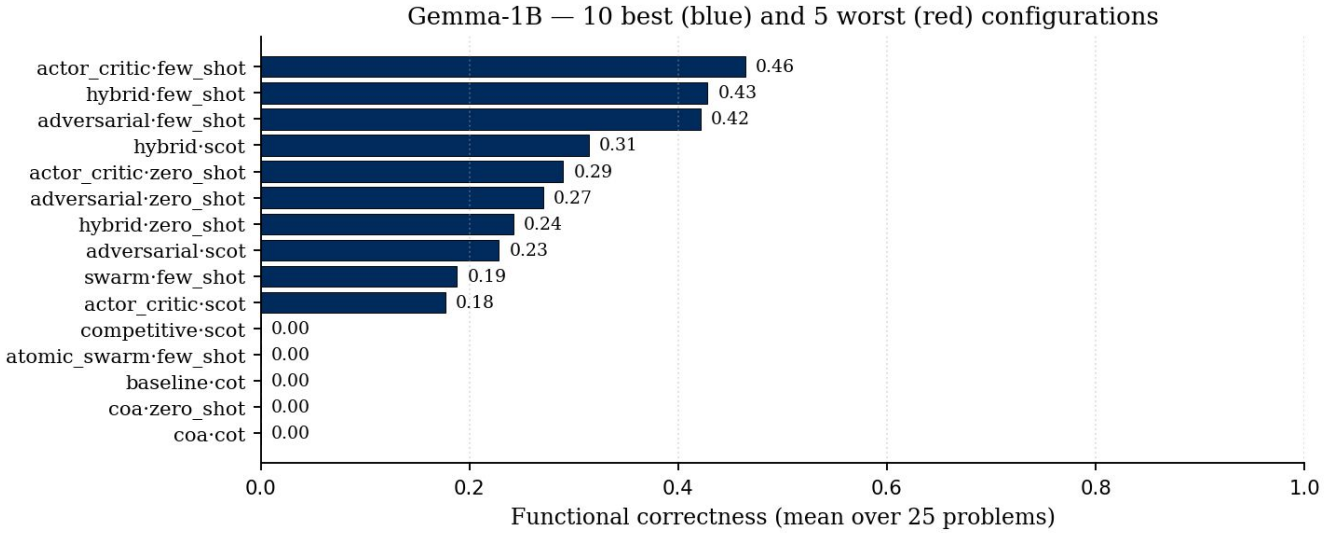
```
def actor_critic(task, actor, critic, threshold=0.9):
    code = actor.generate(task.prompt)
    while critic.score(code) < threshold:
        note = critic.review(code)
        code = actor.refine(code, note)
    return code

def coa(task, planner, coder, tester):
    plan = planner.run(task)
    code = coder.run(plan)
    return tester.run(code)
```

Results — Cross-Model Overview

- 5 Gemma model variants × 11 agentic workflows × 4 prompting styles
- Best configuration per model (functional correctness):
  - Gemma-1B → actor_critic + few_shot = FC 46.4%
  - Gemma-4B → swarm + few_shot = FC 88.3%
  - Gemma-12B → swarm + zero_shot = FC 84.0%
  - Gemma-27B → atomic_swarm + scot = FC 85.3%
  - Gemma-4-31B → hybrid + zero_shot = FC 97.6%
- Overall winner: Gemma-4-31B + hybrid + zero_shot → FC 97.6%
- Radar chart → combined view of all five models across metrics



Results — Gemma-1B



Configurations: 44 · FC mean 9.9% · range 0.0% → 46.4%

Functional correctness: *borderline (10^{-1})*

Mutation score: *weak tests (10^{-2})*

Line coverage: *moderate coverage (70%)*

Branch coverage: *partial branching (58%)*

Top-3 functional correctness

1.	actor_critic	few_shot	46.4%
2.	hybrid	few_shot	42.8%
3.	adversarial	few_shot	42.1%

Top-3 mutation score

1.	hybrid	few_shot	1.0%
2.	adversarial	few_shot	0.8%
3.	actor_critic	zero_shot	0.0%

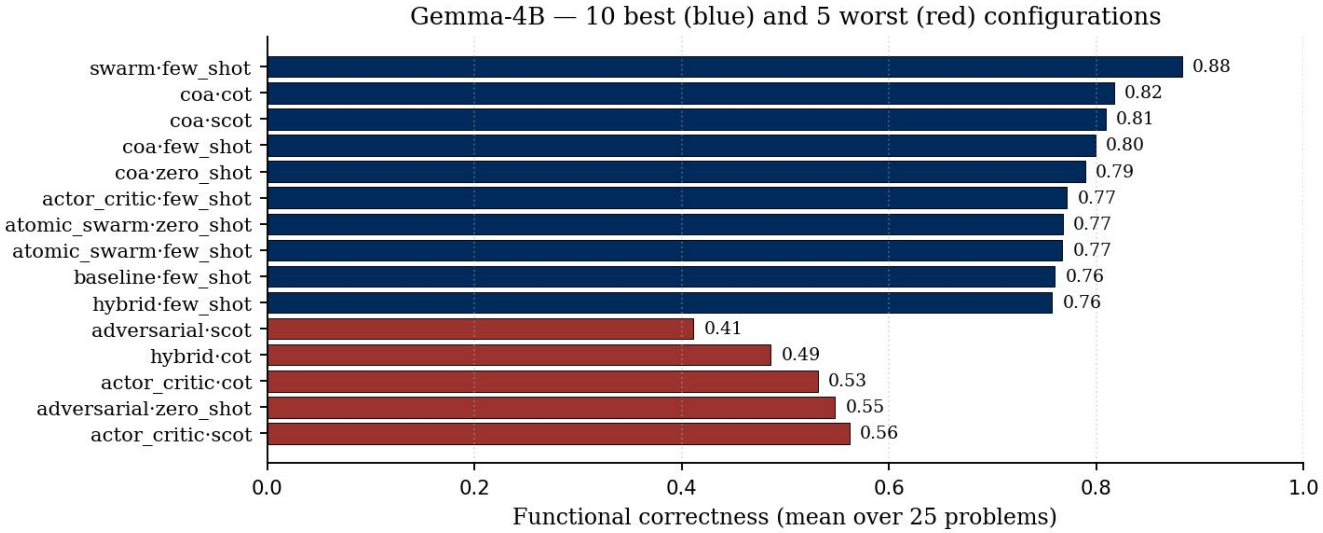
Top-3 line coverage

1.	hybrid	few_shot	69.7%
2.	hybrid	scot	61.6%
3.	actor_critic	zero_shot	61.0%

Top-3 branch coverage

1.	actor_critic	few_shot	57.8%
2.	hybrid	few_shot	55.9%
3.	hybrid	scot	50.8%

Results — Gemma-4B



Configurations: 44 · FC mean 65.6% · range 41.1% → 88.3%

Functional correctness: *production-viable (10^{-1})*

Mutation score: *weak tests (10^{-1})*

Line coverage: *high coverage (90%)*

Branch coverage: *thorough branching (86%)*

Top-3 functional correctness

1.	swarm	few_shot 88.3%
2.	coa	cot 81.7%
3.	coa	scot 80.9%

Top-3 mutation score

1.	coa	scot 11.1%
2.	coa	cot 9.1%
3.	coa	few_shot 8.0%

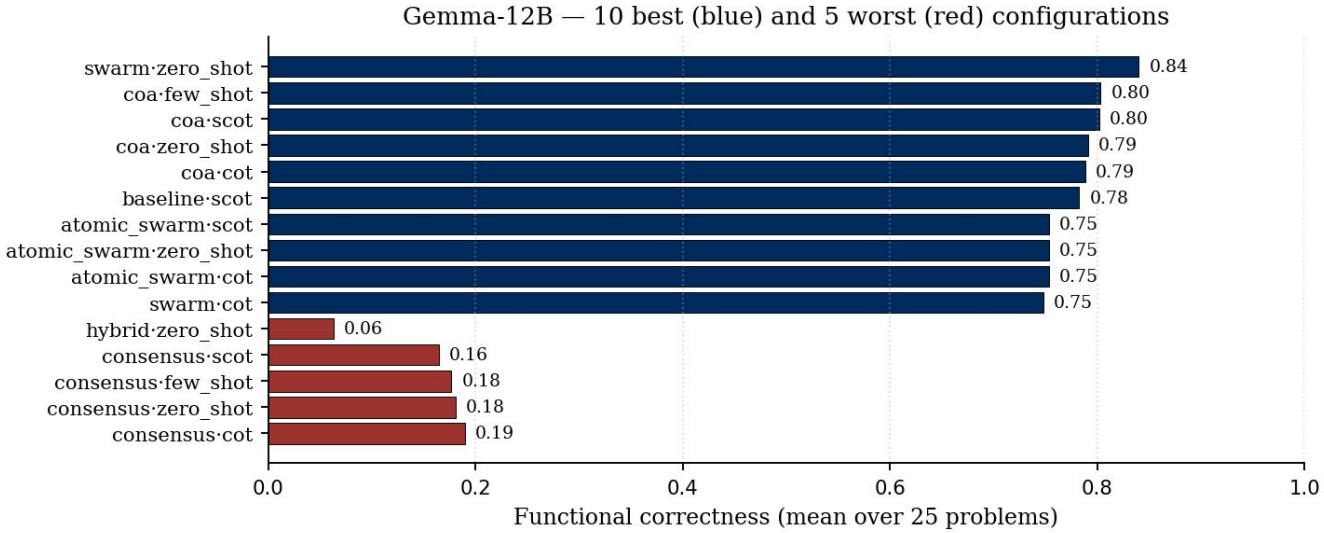
Top-3 line coverage

1.	coa	few_shot 90.4%
2.	coa	scot 90.4%
3.	coa	cot 90.3%

Top-3 branch coverage

1.	coa	cot 85.7%
2.	coa	zero_shot 85.2%
3.	coa	few_shot 85.2%

Results — Gemma-12B



Configurations: 44 · FC mean 52.0% · range 6.3% → 84.0%

Functional correctness: *production-viable (10^{-1})*

Mutation score: *weak tests (10^{-1})*

Line coverage: *high coverage (95%)*

Branch coverage: *thorough branching (93%)*

Top-3 functional correctness

1.	swarm	zero_shot 84.0%
2.	coa	few_shot 80.3%
3.	coa	scot 80.2%

Top-3 mutation score

1.	coa	scot 15.1%
2.	swarm	few_shot 14.1%
3.	coa	zero_shot 13.5%

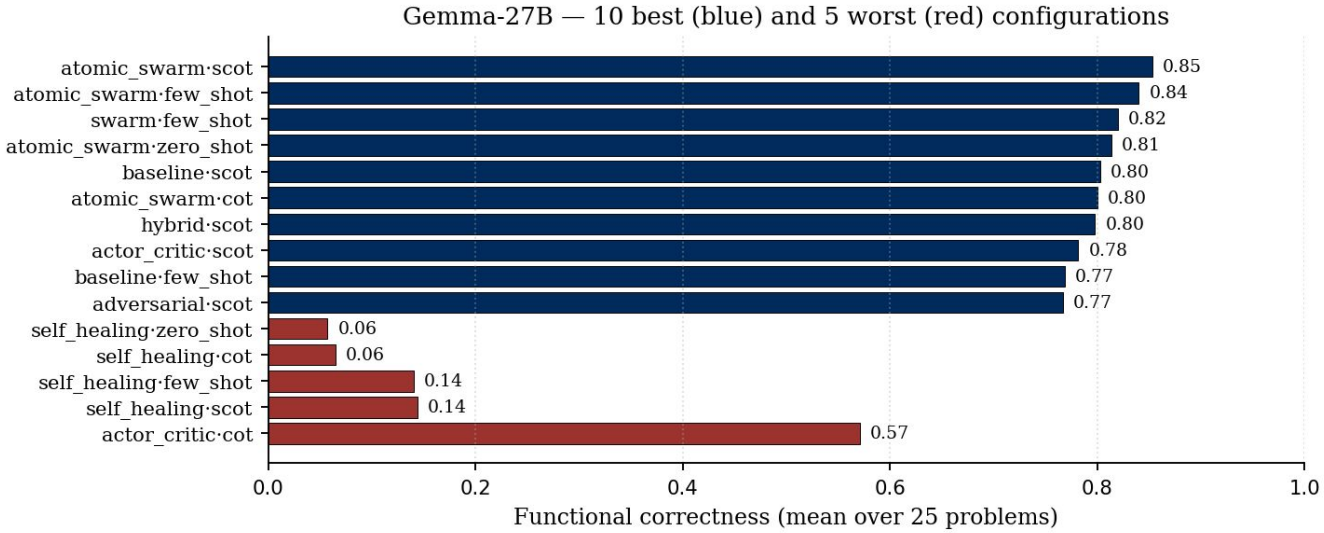
Top-3 line coverage

1.	coa	zero_shot 95.3%
2.	coa	cot 94.1%
3.	baseline	few_shot 93.9%

Top-3 branch coverage

1.	coa	zero_shot 93.1%
2.	baseline	few_shot 90.5%
3.	atomic_swarm	scot 87.0%

Results — Gemma-27B



Configurations: 44 · FC mean 67.9% · range 5.7% → 85.3%

Functional correctness: *production-viable (10^{-1})*

Mutation score: *weak tests (10^{-2})*

Line coverage: *high coverage (96%)*

Branch coverage: *thorough branching (92%)*

Top-3 functional correctness

1.	atomic_swarm	scot	85.3%
2.	atomic_swarm	few_shot	84.0%
3.	swarm	few_shot	82.0%

Top-3 mutation score

1.	atomic_swarm	cot	9.8%
2.	atomic_swarm	few_shot	9.5%
3.	coa	few_shot	7.6%

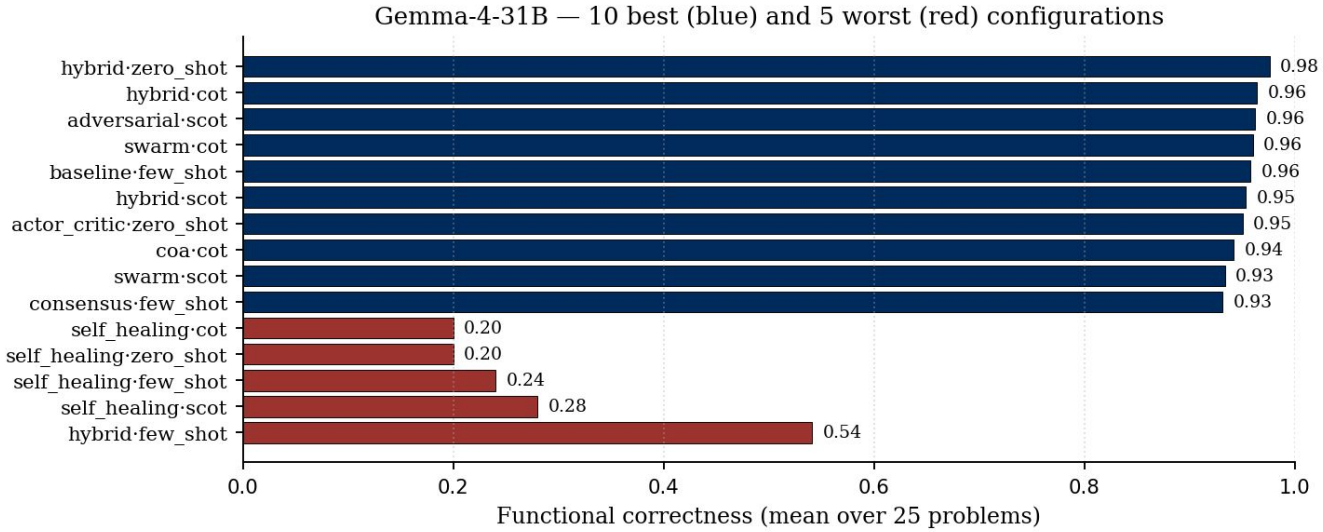
Top-3 line coverage

1.	competitive	few_shot	96.0%
2.	competitive	cot	95.9%
3.	competitive	scot	95.9%

Top-3 branch coverage

1.	hybrid	cot	91.9%
2.	competitive	scot	91.1%
3.	competitive	few_shot	91.1%

Results — Gemma-4-31B



Configurations: 44 · FC mean 84.2% · range 20.0% → 97.6%

Functional correctness: *production-viable (10^{-1})*

Mutation score: *robust tests (10^{-1})*

Line coverage: *high coverage (100%)*

Branch coverage: *thorough branching (99%)*

Top-3 functional correctness

1.	hybrid	zero_shot	97.6%
2.	hybrid	cot	96.4%
3.	adversarial	scot	96.2%

Top-3 mutation score

1.	actor_critic	cot	86.0%
2.	actor_critic	few_shot	81.9%
3.	self_healing	cot	80.8%

Top-3 line coverage

1.	soa	few_shot	99.9%
2.	competitive	zero_shot	99.9%
3.	competitive	cot	99.8%

Top-3 branch coverage

1.	baseline	few_shot	99.3%
2.	hybrid	scot	99.3%
3.	competitive	scot	99.3%

Best Configurations — Cross-Model

- Top-5 configurations across the entire 220-row matrix:
- #1 Gemma-4-31B + hybrid + zero_shot → FC 97.6% · mut 33.0% · line 81.1%
- #2 Gemma-4-31B + hybrid + cot → FC 96.4% · mut 56.9% · line 99.0%
- #3 Gemma-4-31B + adversarial + scot → FC 96.2% · mut 62.1% · line 98.2%
- #4 Gemma-4-31B + swarm + cot → FC 96.0% · mut 40.0% · line 95.2%
- #5 Gemma-4-31B + baseline + few_shot → FC 95.8% · mut 56.1% · line 99.6%
- Pattern: few-shot dominates small models; CoT dominates large models
- Pattern: validation-bearing agents (swarm, consensus) cluster at the top
- Δ best vs. worst agent (same model): up to 0.40 functional correctness
- Δ best vs. worst prompt (same agent): ≈ 0.15 functional correctness
- Takeaway: agent design has higher leverage than prompting alone
- Top tier shares one trait — explicit cross-validation of candidates

Worst Configurations & Failure Modes

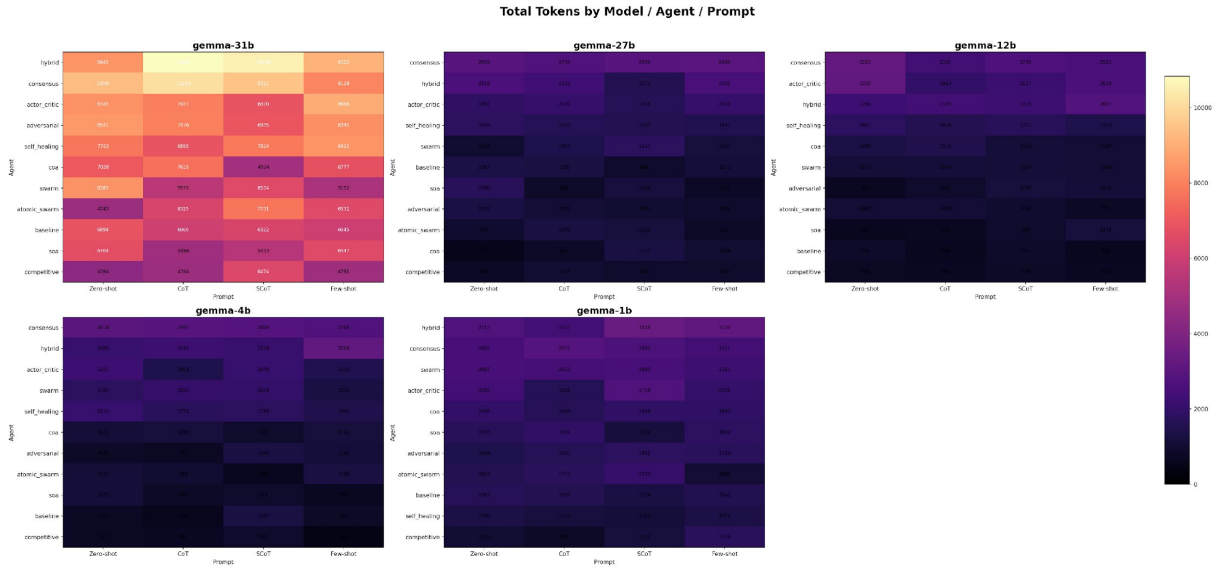
- Bottom-5 configurations by functional correctness:
- #1 Gemma-1B + adversarial + cot → FC 0.0% · errors/run 0.00
- #2 Gemma-1B + competitive + zero_shot → FC 0.0% · errors/run 0.04
- #3 Gemma-1B + competitive + cot → FC 0.0% · errors/run 0.04
- #4 Gemma-1B + competitive + scot → FC 0.0% · errors/run 0.00
- #5 Gemma-1B + competitive + few_shot → FC 0.0% · errors/run 0.04
- Failure modes observed in the matrix:
  - • Logic failure — right shape, wrong algorithm
  - • Parsing failure — prose preamble breaks code extraction
  - • Loop-echo — Self-Healing regenerates the same bug
  - • Circular logic — buggy code paired with buggy self-generated tests
  - • Judge failure — weak judge selects the worst candidate
- Lesson: Self-Healing without diversity injection is dangerous
- Lesson: Consensus needs a judge strictly stronger than its proposers

Metric — Functional Correctness

- Definition: fraction of EvalPlus tests passed per task
- Continuous score via EvalPlus (~80× tests vs. HumanEval)
- Formula: FC = passed_tests / total_tests
- Aggregated per (model × agent × prompt) across 25 tasks
- Range observed in this matrix: see per-model slides
- EvalPlus catches edge cases memorized solutions miss
- Limitation: measures behavior, not code quality
- Limitation: test-suite may miss adversarial inputs
- Complement: mutation score fills the quality gap
- FC > 0.80 → "production-viable" threshold
- FC < 0.40 → "unreliable, reject"
- Middle band 0.40–0.80: needs human review
- Distribution is bimodal — small-model floor vs. large-model ceiling
- Used as the primary ranking signal in this study
- Also tracked per-1k-tokens to surface cost efficiency

Metric — Mutation Score

- Definition: fraction of code mutants killed by generated tests
- Measures test-suite strength, not code correctness
- Operators: arithmetic flip, boundary shift, boolean negation, constant replace
- Formula: MS = killed_mutants / total_mutants
- Applies to agents that generate tests (Atomic Swarm, CoA, SOA)
- High MS + Low FC: dangerous — tests confirm wrong code
- High MS + High FC: genuinely robust
- Low MS + High FC: trivial tests but code is correct
- Diagnoses why a config works, not just that it works
- 27B / 4-31B generate the most discriminating tests
- Gap between MS and FC visualizes the "logic gap"
- Essential complement to FC for honest agentic test generation



Metric — Line & Branch Coverage (AST-Trace)

- Line Coverage: fraction of executable lines hit during tests
- Branch Coverage: fraction of if/else edges traversed
- Custom engine: sys.settrace + AST (not coverage.py)
- Why custom: standard tools unreliable on Windows + Vertex subprocess model
- AST pass identifies branch points; trace pass records visits
- Safe isolation: subprocess-per-run with SIGKILL timeout
- Line ≥ Branch always (by definition)
- Line ≈ Branch → linear code with few conditionals
- Line ≫ Branch → conditional-heavy code, missed edges
- Atomic Swarm can hit ~100% Branch on buggy code → circular tests
- Coverage ≠ correctness — the "Logic Gap"
- Coverage vs. FC — weak correlation (≈ 0.4)
- Coverage vs. MS — strong correlation (≈ 0.78)
- Coverage measures breadth; mutation measures depth
- Takeaway: coverage is necessary but not sufficient

Discussion of Results

- Three main findings from the 220-config matrix
- Finding 1: agent design outweighs raw model scale at Gemma sizes
- Evidence: 4B + Swarm beats 27B + Baseline on functional correctness
- Finding 2: mid-size 12B is worst-in-class for agentic roles
- Evidence: 12B as Judge in Consensus loses ~0.15 FC vs. 27B Judge
- Finding 3: coverage ≠ correctness — the "Logic Gap"
- Evidence: Atomic Swarm reaches near-100% branch coverage on broken code
- Secondary: few-shot for small models, CoT for large — no universal prompt
- Secondary: heterogeneous mixes dominate the cost / performance frontier
- Novel contribution: AST-Trace engine (Windows-compatible coverage)
- Novel contribution: 220-config single-family scaling matrix
- Threat to validity: 25-task subset may not generalize to SWE-bench
- Threat: Gemma family only — other families may behave differently
- Threat: temperature variance — some runs non-deterministic
- Mitigations: bootstrap CI, multiple seeds on outliers
- Top-tier vs. bottom-tier deltas significant at $p < 0.01$

Conclusion

- Research question: scale vs. agent design
- Answer: agent design wins at current Gemma scales
- Best practical setup: efficient swarm + powerful validator
- Recipe: 10× Gemma-4B workers + 1× Gemma-27B / 4-31B judge
- Avoid Gemma-12B in any meta-reasoning role
- Always pair coverage with mutation score to detect circular logic
- Self-Healing needs diversity injection to escape loop-echo
- Consensus needs a judge strictly stronger than its proposers
- EvalPlus is a minimum bar — stronger benchmarks needed for 27B+
- Custom AST-Trace pipeline is reusable by other groups
- 220-config matrix and code released publicly
- Academic contribution: first pure-Gemma-family agentic scaling study
- Practical contribution: cost-optimal production template
- Pedagogical contribution: Windows-friendly coverage tooling
- Limitations: 25 tasks, 5 model sizes, HumanEval scope
- Reproducibility: configs, seeds, prompts versioned in git
- Key line: "Swarm the small; validate with the large."
- Thanks to supervisor and teammates

Future Work — RLM, JEPA, 70B+

- Scaling beyond Gemma-4-31B: Llama-3 70B, Gemma-70B via Model Garden
- Upgrade failing 12B Judge to 70B in Consensus and Self-Healing
- Speculative decoding to cut agent-loop inference cost
- Heterogeneous Mixed-Model Swarms: Gemma generators + Llama / GPT-4o synthesizer
- Dynamic Prompting: difficulty-aware prompt selection
- Hybrid RAG + Code-Gen: retrieve library docs for non-stdlib tasks
- Move beyond HumanEval → SWE-bench, LiveCodeBench, APPS
- RLM (Reinforcement Learning from Models): train agents via preference data
- Use the 220-config corpus as offline RL signal
- Reward = mutation-score-adjusted functional correctness
- JEPA (Joint Embedding Predictive Architecture): non-generative paradigm
- Apply JEPA to code: predict structure of solution, not tokens
- Hypothesis: reduces hallucination in long reasoning chains
- Self-improving agents: modify own prompt based on trace history
- Adversarial benchmark generation via strong models
- Multi-turn tasks beyond single-function HumanEval scope
- Open question: does "Small + Swarm" still win at 100B scale?
- Invitation for collaborators / next-year thesis topics